

Web Application Requirements

Overview

Develop a single-page web application that simulates a movie store showcase by generating fake movie information.

Functional Requirements

LANGUAGE SELECTION

- Must include English (USA).
- Include at least one additional language/region (e.g., German (Germany) or Ukrainian (Ukraine)).

SEED CONFIGURATION

- Allow the user to set a custom seed value (at least 48 bit).
- Provide an option to generate a random seed.

LIKES/REVIEWS PER MOVIE

- The user specifies the average number of likes and reviews per movie.
- Range: 0–10, with fractional values allowed (e.g., 3.7).
- Examples:
 - 0 — no likes for any movie.
 - 0.5 — each movie has either 1 like or 0 likes with 1:1 chances (on average, 1 like for every 2 movies).
 - 10 — each movie has exactly 10 likes.
- Fractional values must be implemented probabilistically to achieve the desired average.

UI / UX

- The controls for language selection, seed configuration, likes-per-movie, reviews-per-movie should be placed horizontally in a single row, functioning as a toolbar.
- Data must update dynamically whenever parameters change, without requiring the user to press Enter or click separate buttons.
- Support two display modes:
 - **Table View** with pagination (no infinite scrolling).
 - **Gallery View** with infinite scrolling (no pagination; in the further text, a "batch" of data in Gallery View is also called a page).
- Any change to generation parameters must:
 - Reset the Table View to the first page.
 - Reset the Gallery View to the initial scroll position.
- Data should be provided to the user **immediately** upon page load, without requiring any input or button presses.
- No hardcoded limit for the data, the users can go to the next page or scroll down as long as they want.

GENERATED DATA

Each table row and each gallery card must contain the following data:

- Sequence index — a sequential number starting from 1 (e.g., 1, 2, 3, ...).
- Movie title — randomly generated.
- Actors — randomly generated personal names.
- Year — randomly generated year.
- Genre — randomly generated genre.
- Trailer (is hidden in Table View, but expandable).
- Reviews and likes (hidden by default in both views, expandable).

AUTHENTICATON

The application must function **without** requiring user registration or authentication.

LANGUAGE INDEPENDENCE

- You are not required to translate the same movie between languages.
- Movies for different languages must be generated independently.

LOCALIZATION RULES

- The language of the movie title, actor names, and genre must match the selected language/region.
- Generated data should look realistic for the chosen language/region (although can be nonsensical, e.g., "Fast Carrot Plus" is OK).
- Placeholder text such as lorem ipsum is **not** acceptable.

PARAMETER INDEPENDENCE

- All parameters — region, seed, likes and reviews — are independent.
- Changing one parameter does not reset the others, but:
 - Changing likes only updates the like counts; titles, actors, etc. stay the same.
 - Changing review count only add or remove the reviews; titles, actors, etc. stay the same.
 - Changing seed or region changes titles, actors, genres, review texts and trailers (everything must update dynamically in response to parameter changes).

TABLE VIEW

Each record in the Table View must be expandable. The expanded view should be collapsible so the table returns to its original state without a full page reload.

When the user clicks on a record, detailed information is displayed, including:

- Trailer freeze frame — randomly generated, with the correct title rendered on the frame.
- Play button — allows playback of a short trailer.
- Some randomly generated review text.s

TRAILER GENERATION

- You must generate random trailer that can be played directly in the browser.
- Movies must be reproducible: the same seed values must always generate the same text/video output.

OPTIONAL REQUIREMENTS

- Add an Export button that generates a ZIP archive containing video files for the current page in the Table View. Each file should be named using the corresponding movies title.
- Add series with seasons to the app.

TRAILERS

The most important part of the task is trailers.

The better they will look (and sound), the higher the grade. Yes, it can be a challenge to find the completely **free** and **fast** video generation service, but you may use the combined approach.

The length of the trailer shouldn't be too long, 5-10 seconds is enough.

It's probably not the best possible variant, but you may:

- Generate a typographic animation of the movie title (note that you don't technically need to render it into a movie clip; if you preload the rest, you may show animation on the canvas and do a transition to the video). Choose an animation and its parameters randomly.

- Generate an alternative typographic animations for “THIS FALL”, “JOHN SMITH JR.” and the likes. Choose animations and their parameters randomly.
- Select randomly two or three short pre-rendered clips. Note that having 5 or 10 different video won’t suffice, but $500 * 500$ is = 250000, right? Apply random color correction effects, randomly adjust zoom and speed.
- Combine everything with some randomly selected transitions.

And you’ve got yourself a trailer.

Of course, you you manage to actually generate the whole clip, kudos!

The sound isn’t required, but a good movie trailer would benefit from a music or/and thunder and blast effects.

Implementation Notes

REGION-SPECIFIC DATA

Your code must **not** contain any hardcoded region-specific values, words, or other data. All locale-specific content must come from external resources (e.g., lookup tables, configuration files, or localization datasets) so that languages and regions can be added, removed, or updated without modifying the source code.

Use 3rd-party libraries for the fake data generation.

ARCHITECTURE

All data must be generated on the server side, not in the browser (although the data itself can be some animation sequences to run in the browser; but all the generations from the seed should be performed on the server).

You do not store generated random data in the database, of course.

The browser connects to a single server that provides a single page (a batch) of data generated in memory. No database is required for storing random data (though you may use a database for lookup tables if needed).

SEEDS

When the user changes the seed, the generated data must update accordingly. The seed provided to the RNG algorithm should be a combination of the user-specified seed and the displayed page number. The exact method of combining the seed and page number is not critical; a simple MAD operation is sufficient. **Important:** the same seed, when entered at any time, must produce **identical** data, ensuring reproducibility across devices and dates.

Changing the seed must change the generated data, but the same seed must always produce the same results. Adjusting the number of likes must not alter generated titles, actors, or other core content—these should depend solely on the seed value and the record index.

CODE REUSE

You are required to use third-party libraries for random data generation (e.g., some port of Faker) and **may need to extend these libraries** in ways supported by their APIs. It is expected that you will also use appropriate libraries for other tasks such as video generation, implementing infinite scrolling, and other required functionality.

Additional notes

- Data should look **as realistic as possible**, though it does not need to be semantically meaningful.
- Trailers must have movie title inside as well as some video that look like, well, video, random rotating lines and/or gradients will get a low grade.
- The video content must have significant variations, 2 or 3 rolls won't be enough.
- Note that higher grades will be given to titles and trailers that appear visually appealing and realistic, with coherent animated titles, melodies and rhythms.
- AI services may also be used to enhance the generation if desired. If you enjoy working with AI agents, feel free to leverage them! *Typically, you may expect better results if AI generates data, not code that generates data.*